

Building with the Claude API — A Conceptual Textbook

This is a from-scratch conceptual reference for everything covered in Anthropic's "*Building with the Claude API*" course. It exists independently of the exercises in `exercises/` — you don't need to have done any of them to read this, and this document occasionally points at one by name as a worked example, but the goal here is to explain the *ideas*, not to walk through any particular implementation. Code in this document is illustrative pseudocode (mostly JSON shapes), not runnable JavaScript.

The chapters build on each other in order: Part I covers the request/response shape every later feature reuses, Part II teaches you how to *measure* whether a prompt works instead of guessing, Part III covers giving Claude the ability to act instead of just talk, Part IV covers grounding Claude in documents too large to fit in a prompt, Part V covers a set of largely independent advanced capabilities, and Part VI zooms out to how you compose everything above into a real application.

Table of Contents

- **Part I — Foundations of the Messages API**
 1. [The Messages API Is a Stateless Completion Primitive](#)
 2. [Conversations, History, and System Prompts](#)
 3. [Streaming](#)
 4. [Structured Output: Prefill and Stop Sequences](#)
- **Part II — Evaluating and Engineering Prompts**
 5. [Why Systematic Evaluation Matters](#)
 6. [Building a Test Dataset](#)
 7. [The Eval Pipeline as an Architecture](#)
 8. [Grading Strategies](#)
 9. [The Iterative Prompt Engineering Cycle](#)
- **Part III — Tool Use (Function Calling)**
 10. [Why Tools Exist](#)
 11. [The Round Trip](#)
 12. [Tool Schemas](#)
 13. [Claude's Decision to Use a Tool](#)
 14. [The Response Shape: Content Blocks and `stop\_reason`](#)
 15. [Sending Results Back: `tool\_result`](#)
 16. [Multiple Tools and the Agentic Loop](#)

- 17. [Multi-Turn Conversations With Tools](#)
 - 18. [Streaming Tool Use](#)
 - 19. [Built-in Tools vs. Custom Tools](#)
 - **Part IV — Retrieval-Augmented Generation (RAG)**
 - 20. [Why RAG Exists](#)
 - 21. [Chunking Strategies](#)
 - 22. [Embeddings and Semantic Search](#)
 - 23. [The Full RAG Pipeline, Worked Numerically](#)
 - 24. [Vector Indexes](#)
 - 25. [Lexical Search and BM25](#)
 - 26. [Hybrid Search and Reciprocal Rank Fusion](#)
 - **Part V — Advanced Capabilities**
 - 27. [Extended Thinking](#)
 - 28. [Vision \(Image Analysis\)](#)
 - 29. [PDF Processing](#)
 - 30. [Citations](#)
 - 31. [Prompt Caching](#)
 - 32. [Code Execution](#)
 - 33. [The Files API](#)
 - **Part VI — Agents and Workflows**
 - 34. [Workflows vs. Agents](#)
 - 35. [Workflow Pattern: Evaluator-Optimizer](#)
 - 36. [Workflow Pattern: Parallelization](#)
 - 37. [Workflow Pattern: Chaining](#)
 - 38. [Workflow Pattern: Routing](#)
 - 39. [Agents and Tools](#)
 - 40. [Environment Inspection](#)
 - 41. [Choosing Between Workflows and Agents](#)
 - [Closing: How the Pieces Fit Together](#)
-

Part I — Foundations of the Messages API

1. The Messages API Is a Stateless Completion Primitive

Every capability covered in this book — streaming, tool use, RAG, vision, agents — is a variation on one underlying primitive: you send Claude a complete snapshot of a conversation, and it returns exactly one new turn. Understanding this primitive deeply

pays off everywhere else, because nothing about the mental model changes later — only the *contents* of what gets sent and returned grows richer.

The API is stateless. There is no server-side session, no login state, no memory of a previous call held anywhere by the model. Each request stands entirely on its own — Claude only knows what's physically present in that one request's payload. This is the single fact underneath almost every other concept in this chapter and the next: "having a conversation" with Claude is an illusion the *client* maintains by resending history, not something the server does for you.

Anatomy of a request

Field	Purpose
model	Which Claude model handles this request.
max_tokens	A cap on the reply, not a target — the model routinely stops well short of it. This is a common misconception worth correcting early: it bounds cost/length risk, it doesn't force verbosity up to the limit.
messages	An array of turns, { role, content } each. This array is the conversation as far as the API is concerned — nothing outside it persists between calls.
system	A separate, top-level field for standing instructions (see Chapter 2) — not a message, not part of messages.
stop_sequences	Strings that immediately end generation if produced (see Chapter 4).
stream	Switches delivery mechanics to incremental events (see Chapter 3) — doesn't change the semantic content of the reply.

Anatomy of a response

The reply comes back as a `content` array, never a plain string:

```
{
  "role": "assistant",
  "content": [
    { "type": "text", "text": "... " }
  ],
  "stop_reason": "end_turn",
```

```
"usage": { "input_tokens": 42, "output_tokens": 17 }
}
```

This array shape exists because a single reply can legitimately mix several kinds of content in sequence — reasoning followed by an answer, or an answer followed by a request to call a tool. The idiomatic way to consume a response, from the very first exercise in the course onward, is to **iterate the content array and branch on each block's type** — never assume the reply is "block 0" or a bare string. Block types you'll meet across this book: `text` , `tool_use` , `tool_result` , `thinking` , `redacted_thinking` , `document / image` (as request-side blocks), and various server-tool blocks.

Two other response fields worth knowing before you need them:

- **stop_reason** — why generation stopped: `"end_turn"` (Claude finished naturally), `"max_tokens"` (hit the cap), `"tool_use"` (paused, waiting on a tool result — Chapter 14), `"stop_sequence"` (hit a configured stop string — Chapter 4).
- **usage** — token accounting: `input_tokens` , `output_tokens` , and (once prompt caching is in play — Chapter 31) `cache_creation_input_tokens` / `cache_read_input_tokens` .

Roles and turn structure

Every entry in `messages` carries a `role` , restricted in normal use to `user` and `assistant` . Turns are expected to **strictly alternate** — `user` → `assistant` → `user` → ... Breaking this (two `user` turns in a row, or forgetting to append the assistant's reply before sending the next user turn) causes the next call to fail or behave oddly. This constraint resurfaces constantly once tool use is added, because a tool result is technically sent as a `user` turn (Chapter 15) — it's easy to mis-thread this and accidentally record Claude's own reply under the wrong role.

There is deliberately no role for "how should you behave overall" framing — that's exactly the gap the `system` parameter exists to fill (Chapter 2).

Key discipline for round-tripping Claude's own reply: when you feed a prior assistant turn back into `messages` for the next call, push the raw `content` array Claude actually returned — not just the text you extracted from it. Early on, when only `text` blocks exist, this looks like a distinction without a difference. It becomes load-bearing the moment richer block types (`tool_use` , `thinking`) enter the picture: extracting only the text silently drops information the next turn needs to make sense of what already happened.

2. Conversations, History, and System Prompts

Multi-turn conversations are client-side bookkeeping

Because the API is stateless (Chapter 1), a real back-and-forth conversation is implemented entirely by the client: each request resends the *entire* transcript so far, not just the newest line. The canonical shape is an ever-growing array, mutated across loop iterations:

1. Push the new `user` turn onto the history array.
2. Call the API with the full array.
3. Push the resulting `assistant` turn (the full content-block array — see Chapter 1) back onto the same array.
4. Repeat.

Two limitations are worth internalizing early, because they don't go away — they're just deferred:

- **Unbounded growth.** Every turn resends everything before it, so token usage — and therefore cost and latency — climbs monotonically as a conversation lengthens. Real applications eventually need a trimming/summarization strategy.
- **No persistence across process restarts.** The history array typically lives only in memory; restarting the program loses the conversation unless you explicitly save/restore it somewhere.

System prompts: a separate channel for standing instructions

The `system` parameter is a distinct, top-level field — a sibling of `messages`, not a message itself and not stored as a turn. It exists because nothing in the `user / assistant` pair has room for instructions about *how to behave for the whole conversation* (persona, tone, constraints, refusal rules) — those need a channel that isn't itself part of the turn-by-turn dialogue.

- **Resent every call, same as everything else.** Because the whole request is stateless, there's no "set once" semantics — a system prompt must be included on every call to stay in force. This is a direct corollary of statelessness, not a separate mechanism.
- **Its scope is broader than tone.** A system prompt can shape problem-solving strategy and refusal behavior — e.g. a math-tutor persona that's forbidden from giving final answers and must guide step-by-step instead — which means it can override what a user directly and explicitly asks for.

- **Its value is usually a plain string**, but it can also be an array of content blocks for more advanced use (this becomes relevant for prompt caching — Chapter 31, where individual blocks need their own cache markers).
- **It's optional** — a request is valid with or without one.

3. Streaming

Why streaming exists

Without streaming, a call **blocks until the entire reply is generated**, then returns it as one object — for a long generation, that means no output at all until everything is done. Streaming instead opens a connection where the server pushes incremental events as generation happens, which buys you:

- **Better UX** — displaying partial text as it's produced (typewriter-style output) instead of a long silent wait.
- **Avoiding timeouts** — a large `max_tokens` non-streaming call risks hitting an HTTP timeout before the full response completes; streaming sidesteps this by delivering continuously.

A stream is a sequence of typed events, not one payload

Each event has a `type` and its own data shape. For a simple text response, in order:

```
message_start
  → content_block_start (index 0, type: "text")
  → content_block_delta (index 0, delta: { type: "text_delta", text: "Hi" })
  → content_block_delta (index 0, delta: { type: "text_delta", text: "!" })
  → content_block_stop (index 0)
message_delta (final stop_reason, final usage)
message_stop
```

Event	Meaning
message_start	Response begins; carries an initial, mostly-empty message shell plus input token usage.
content_block_start	A new content block begins at a given <code>index</code> ; this is where the block's <code>type</code> is established.
content_block_delta	Incremental payload for whatever block is currently open — its meaning depends entirely on the delta's own sub-type (see below).

content_block_stop	The currently-streaming block is complete.
message_delta	Top-level fields that only become final at the very end (stop_reason, usage) — they arrive late because they genuinely aren't known until generation is essentially done.
message_stop	The entire response is fully finished.

content_block_delta sub-types you'll encounter across this book: text_delta (visible reply text), thinking_delta / signature_delta (extended-thinking output, Chapter 27), input_json_delta (a chunk of a tool call's arguments, Chapter 18), citations_delta (a citation attached to streamed text, Chapter 30).

A response can contain multiple content blocks in sequence — e.g. a thinking block followed by a text block, or several tool-call blocks in a row — each getting its own content_block_start / ..._delta(s) / content_block_stop cycle at its own index. Streaming exposes this multi-block structure explicitly; a non-streaming call just hands back the already-finished array.

Two ways to consume a stream

- **Raw event access** — iterate every event as it arrives, seeing the full taxonomy above. Necessary for anything beyond plain text: tool-call arguments, thinking, citations.
- **Filtered convenience access** — a higher-level facility that watches the raw stream internally and emits only one specific, already-unwrapped payload (e.g., a plain string, emitted only for text deltas). Pulling any one specific thing out of a mixed stream is always a two-level check (is this a content-block delta? *and* is the delta's own type the one I want?) — convenience layers exist precisely so application code doesn't reimplement that check by hand.

Once a stream fully ends, it exposes a way to await the **complete final message** — the exact same shape a non-streaming call would have returned (full content array, final stop_reason, final usage), just resolved asynchronously once the terminal event has arrived.

Gotchas

- **Don't assume content_block_delta always means text.** Its meaning is entirely conditional on which block is open and the delta's own sub-type —

treating every delta as text-shaped breaks the moment thinking or tool-use blocks are involved.

- **A stream can carry multiple, differently-typed blocks in one response** — code that assumes "one block per response" will misbehave.
-

4. Structured Output: Prefill and Stop Sequences

The problem

By default, asking Claude for a strictly-parseable format (JSON, CSV) yields **prose wrapped around the data** — an intro sentence, a fenced code block, often a trailing explanation. Fine for a chat UI; it breaks any caller that wants to feed the raw output straight into a parser.

Prefilling the assistant turn

Instead of ending `messages` on a `user` turn (the normal case), you end it on an `assistant` turn containing partial content you choose — e.g. an opening code-fence marker, or a full lead-in sentence. Claude then **continues generation from wherever the conversation appears to leave off** — it treats the prefilled text as if it had already written it, and produces only the continuation. Concretely, it no longer re-decides whether to add an introductory sentence or wrap the answer in commentary, because as far as it's concerned that decision point has already passed.

The prefill can be as small as an opening fence delimiter, or as large as a full sentence that asserts its own constraints ("Here are exactly three ideas, one per line, with no commentary:") — the model then behaves consistently with a commitment it believes it already made. This generalizes beyond syntax markers: *any* text that plausibly frames the response can be used as a prefill.

Stop sequences

`stop_sequences` is an array of exact strings that, if generated, halt generation immediately (the sequence itself isn't included in the returned text). Two distinct roles show up for the same mechanism:

1. **Marking literal end-of-content** — e.g. using the closing fence marker as the stop sequence, so generation halts the instant Claude tries to close the fence it was prefilled to open, isolating exactly the payload between the two fences.
2. **Guarding a count** — e.g. prefilling a numbered list's start and setting the *next* number as the stop sequence, cutting generation off before an unwanted extra

item — a structural upper bound on quantity, independent of whatever the prefill prose asked for.

Limits, worth internalizing

- **Prefill + stop-by-fence does not guarantee a requested quantity.** Prefilling a sentence that promises "three items" mostly produces three, but not reliably — nothing about a content-based stop sequence structurally enforces a *minimum* count.
- A **counting-convention stop sequence** gives a hard guarantee on the *upper* bound only — neither technique alone guarantees both bounds.
- Output still typically needs post-processing (trimming a leftover leading newline before `JSON.parse`, etc.) — treat anything nominally structured coming back from the model as needing defensive handling, not a guaranteed clean payload.

A currency note: this technique is being phased out

Assistant message prefill has been removed on current-generation models — it's rejected outright (HTTP 400) rather than silently degrading. This matters because prefill + stop sequence is exactly the kind of technique you'll find in older course material (this course itself was originally taught in Python against older models), and it's worth understanding conceptually — but for new work against current models, reach for the modern replacements instead:

- For structured/JSON output specifically: a dedicated **structured outputs** mechanism (attach a schema to the request so output is constrained/validated directly), or tool definitions with a strict mode enforcing their argument schema.
- For other output shaping: plain **system-prompt instructions** ("respond with only X, no commentary") rather than trying to trick the model via conversational framing.

This is a recurring theme worth flagging once, here, so it doesn't need repeating: **API capabilities drift across model generations**. A technique that works today may be rejected on tomorrow's flagship model, and vice versa — always sanity-check an older technique against whatever model you actually target.

Part II — Evaluating and Engineering

Prompts

5. Why Systematic Evaluation Matters

Prompt engineering (Chapter 9) is the craft of *writing* a better prompt. **Prompt evaluation** is a different, complementary discipline: it doesn't touch how a prompt is written, it *measures* whether a prompt — however it's written — actually works, by testing it against a dataset of representative inputs and scoring the results. Evaluation is the measuring stick; engineering is what you do with the number it produces.

After drafting a prompt, there are three paths forward:

1. **Test it once and ship it.** Fast, but breaks on the first unanticipated real input.
2. **Test it a few times, patch the corner cases you happened to notice.** The trap most engineers fall into by default, not through carelessness — real-world input variety always exceeds what you thought to try.
3. **Run it through an eval pipeline, score it objectively, iterate on the score.** More upfront cost, but the only path that gives confidence the prompt holds up outside your own manual testing.

This chapter and the next three build up path 3.

6. Building a Test Dataset

A **dataset** is an array of task/scenario descriptions run through the prompt-under-test one at a time, so performance can be inspected across a spread of cases instead of eyeballing a single example — a handful of cases for a first pass, scaling to hundreds or thousands for a mature eval.

A single **test case** accumulates fields as grading needs grow:

- `task` — the description of what's being asked (the core input).
- `solution_criteria` — **what a correct solution to this task must include**: a concrete, checkable rubric, explicitly *not* vague praise like "well-written." This is what turns "how good is this?" from a vague question into a checkable judgment.
- `format` — e.g. "python" / "json" / "regex", so a code grader (Chapter 8) knows which deterministic parser/validator to run per case.
- `prompt_inputs` — concrete values to interpolate into the prompt template.

Generating a dataset with the model itself

Use a **cheap, fast model** for dataset generation — it isn't the thing being measured, so it doesn't warrant the strongest, most expensive model. Save that for the prompt actually under evaluation. The classic technique here is exactly the prefill + stop-sequence pattern from Chapter 4 (an opening ````json` fence prefilled, a closing fence as the stop sequence), used to trap the reply to exactly the structured content without manual string-stripping.

A more mature pattern splits generation into **two steps**:

1. Generate a batch of short, distinct **scenario ideas** in one call.
2. Expand **each idea individually** into a full test case (concrete inputs
 - solution criteria) in a separate call per idea.

Asking for N complete test cases in a single shot tends to produce "variations on a theme" — low diversity. Splitting the *decision* (what scenarios to test) from the *elaboration* (fleshing one out) yields more genuinely distinct scenarios.

7. The Eval Pipeline as an Architecture

Conceptually, an eval pipeline is three nested, deliberately ignorant responsibilities:

```
run-the-eval      (loops over the whole dataset)
└─ run-one-case   (runs the prompt, then grades the output)
    └─ run-the-prompt (merges one test case into the template, calls the model)
```

- **Run-the-prompt** knows nothing about grading or iteration.
- **Run-one-case** knows nothing about the dataset as a whole.
- **Run-the-eval** knows nothing about prompts or grading internals.

Why the separation matters: it makes grading a drop-in replacement at the run-one-case layer only. Swapping a placeholder score for a real grader (or combining multiple graders — Chapter 8) never requires touching the prompt-running or dataset-looping logic.

A **result record** per test case carries, at minimum, the model's output, the original test case, and a score (later extended with grader reasoning). Establishing this shape early — even with a fake hardcoded score — is valuable in itself: it proves the pipeline plumbing works end-to-end before real grading logic exists.

Execution: sequential vs. concurrent

A simple eval runs test cases one at a time. A mature one runs them **concurrently, capped at a concurrency limit** — necessary because the work is I/O-bound (network calls) and uncapped concurrency risks hitting API rate limits. The practical dial: keep concurrency and dataset size low while mid-iteration (faster feedback loop), and turn both up only for a final, higher-confidence validation pass once you're satisfied with a prompt.

The naive baseline is intentional

An intentionally bare starting prompt (no formatting instructions, no system prompt) is used precisely because the eval's job is to reveal how bad it is. "Fixing" the prompt is not something to do inside the eval pipeline itself — that's the job of the iterative loop in Chapter 9.

8. Grading Strategies

Three kinds of graders

Grader	How it works	Strength	Limit
Code grader	Deterministic programmatic check (length, banned words, valid syntax, format compliance)	Cheap, fast, fully deterministic	Only works for properties expressible as a fixed rule
Model grader	A second, independent Claude call judges the first call's output	Flexible enough to judge fuzzy/nuanced criteria (quality, completeness, tone, "did this solve the task")	Costs money, adds latency, and is itself imperfect/noisy
Human grader	A person judges the output	Most flexible	Most expensive; reserved for judgment calls no automated check can approximate

Decision rule: if a property has one correct answer checkable with a parser/rule (does this parse, is this valid JSON, is the output free of banned words), use a **code grader** — using a model for this is strictly worse: slower, costlier, and no more reliable than just trying to parse it. If the property is inherently fuzzy or subjective, use a **model grader**. Real evals often use both, on the same output, for different criteria, and combine the scores.

What a model grader looks like

- A **second, independent Claude call** — a completely fresh conversation, not a continuation of the one being judged.
- Given the original task, the output being judged, and the rubric to judge it against.
- **Structured with XML-style tags** for each distinct input
(`<task>` / `<solution>` / `<criteria>`) — the same delimiting idea used in prompt engineering generally (Chapter 9), applied here so a long or code-heavy output can't be misread as blurring into the next section.
- **Ask for more than a bare score.** Models asked for a bare number tend to cluster in a narrow, noncommittal band (often around 6). Requesting structured output — strengths, weaknesses, reasoning, *and* a numeric score — forces the model to commit to an actual judgment rather than guessing a number in isolation, and the reasoning is also useful to a human skimming *why* a case scored low.
- **Mandatory/gating criteria:** some requirements shouldn't be scored on a sliding scale — a violation caps the score at a low ceiling regardless of how well the rest of the output performs, distinguishing non-negotiable requirements from softer, case-specific quality criteria.

Model graders are noisy, not oracles. Re-running the same eval produces a different average score each time, the same nondeterminism affecting any non-zero-temperature call. Treat a model grader's score as a useful, consistent-*enough* signal for comparing prompt versions — never as ground truth for any single run.

Composite scoring

When both a model grader and code grader(s) apply to the same output, combine into one score. An unweighted average is a reasonable starting point, but real evals often weight unevenly — e.g. weighting a hard failure like broken syntax more heavily than a softer quality dimension, since broken output is a failure regardless of how well-intentioned the content is. Tune weighting based on what the eval reveals, not upfront guesswork.

A practical pitfall: parsing model-generated JSON

Asking a model to emit JSON doesn't guarantee *valid* JSON — it's still generative output. A concrete, systematic failure mode: when a test case involves regex-like content, a grader's own explanatory text tends to quote patterns verbatim with unescaped backslashes, producing invalid JSON. Since this is systematic (tied to the content), blind retry alone isn't reliable — a fresh sample can hit the identical failure. The robust fix is sanitizing/ escaping the response before parsing, with retry kept only as a backstop for non-systematic failures. General lesson: **treat any model output nominally in a structured format as an external boundary requiring defensive parsing**, not a guaranteed-valid payload.

The single most important takeaway of this chapter

A score's value is relative, not absolute. A 7/10 or 8/10 tells you almost nothing in isolation — what matters is comparing the *same dataset* scored against two different prompt versions. This point holds for both model grading and code grading, and it's the foundation the entire iterative loop in the next chapter is built on.

9. The Iterative Prompt Engineering Cycle

The loop

1. **Set a goal** — what should the prompt accomplish?
2. **Write an initial prompt** — deliberately naive on the first pass.
3. **Evaluate it** — run it through the eval pipeline (Chapters 6–8).
4. **Apply one prompt-engineering technique** — one concrete, specific change.
5. **Re-evaluate** — confirm the change actually improved the score before keeping it. Repeat steps 4–5 until the score is "good enough" — a convergence loop, not a fixed number of iterations.

Change one thing at a time. If you change three things and the score improves, you can't attribute the improvement — one change might help while another hurts, and net movement can mask that.

Don't be discouraged by a low baseline

A naive first prompt — bare question, no structure, no guidance, no examples — routinely scores very low against a rigorous grader. **This is expected, not a sign something is broken:** the number only becomes useful once you have a second number, from a revised prompt, to compare against. A representative progression: a

naive prompt might score ~2/10; being clear and direct might bring it to ~4/10; adding output guidelines might roughly double that again to ~8/10. Treat this as a rough prior for where the biggest jumps tend to come from, not a target to match exactly.

Four techniques, in recommended order

1. Be clear and direct. The opening line of a prompt does the most work — it's where the model decides what task it's even doing. Use plain language, name what you want directly, and phrase as an instruction with an action verb ("Generate," "Write," "Identify") rather than a question or a soft lead-in ("I was wondering..."). Try this first — later techniques assume the model already understands the basic task.

2. Be specific. Unconstrained prompts force the model to guess at things you actually care about. Two distinct sub-kinds:

- **Output quality guidelines** — properties the *finished* answer must have (length, structure, required elements, tone), checked after the fact. Near-universal, cheap insurance — often the single highest-leverage technique (a representative jump: doubling a score by adding guidelines alone).
- **Process steps** — a sequence the model works through *before* answering (brainstorm options, pick the best, then write it up). Reserved for tasks that benefit from considering multiple angles first (troubleshooting, decision-making) — not needed for straightforward generation.

3. Structure input with tags. When a prompt mixes instructions with multiple distinct pieces of real data, wrapping each in a descriptive tag (invented tags are fine — `<athlete_information>` rather than a generic `<data>`) removes the ambiguity of where one section ends and another begins. The same idea used in grading prompts (Chapter 8). Payoff scales with complexity — modest for a short, simple input; much larger as interpolated content grows bigger or more heterogeneous.

4. Provide examples (one-shot / multi-shot). Showing a sample input paired with an ideal output often teaches format, tone, or edge-case handling more reliably than describing it in prose — the canonical case is sarcasm in sentiment classification, correctly caught only once an example demonstrates the pattern. Best practices: wrap each pair in its own tags, state plainly that an example follows, explain *why* the example output is good (the reasoning transfers better than the raw example alone), and mine your own eval's highest-scoring real outputs as examples rather than hand-writing them from scratch. This is the **most expensive technique to write and maintain**, so it's typically the last one reached for.

Organizational patterns worth carrying forward

- **Model the evaluator as a stateful object** once it needs shared configuration (a concurrency cap) used across multiple operations (generate a dataset, run the eval, grade an output).
 - **Separate infrastructure from the thing being iterated on.** The eval/ grading machinery is stable scaffolding; the prompt under test is the one piece meant to change repeatedly — keep it easy to find, isolated from the scaffolding.
 - **Treat reports as a first-class output**, not just a raw results array — aggregate stats plus per-case detail (visually flagged by score band) let a person quickly diagnose *why* something scored low.
 - **Preserve prior runs.** Since the whole point is before/after comparison, a fresh eval run shouldn't silently overwrite the previous one — some form of archiving is necessary to actually support the comparison the workflow depends on.
-

Part III — Tool Use (Function Calling)

10. Why Tools Exist

Claude's knowledge is frozen at training time and it has no live state: no clock, no current events, no access to external systems or a filesystem, no way to *act* — only to talk. **Tool use** bridges that gap: you describe a function (a "tool") to Claude, Claude decides *when* it's relevant and *what arguments* to call it with, your own code executes it, and the result is fed back so Claude can finish its answer grounded in fresh information instead of guessing or refusing.

A useful running example: a reminders assistant needs to answer "remind me about my doctor's appointment a week from Thursday." That single request exposes three separate capability gaps, each solved by its own tool:

- No time awareness → a `get_current_datetime` tool.
- Unreliable date arithmetic → an `add_duration_to_datetime` tool.
- No reminder mechanism → a `set_reminder` tool.

This unit is the largest in the course because it's introduced incrementally — schema, then a single call, then the result round-trip, then multiple tools, then multi-turn, then streaming, then Anthropic's own built-in tools — deliberately isolating each mechanic before layering on the next.

11. The Round Trip

1. **Initial request** — send Claude a question *plus* tool definitions.
2. **Tool request** — Claude decides it needs a tool and replies with a `tool_use` content block naming the tool and its arguments.
3. **Data retrieval** — your own code runs the matching function. Claude never executes custom tools itself.
4. **Result delivery** — the function's output is sent back as a `tool_result` block.
5. **Final response** — Claude answers using the result (or asks for another tool — see Chapter 16).

A useful five-step checklist for building a tool from scratch: (1) write a plain function, (2) write a schema describing it, (3) call Claude passing the schema via `tools`, (4) run whatever tool Claude asked for, (5) send the result back and call Claude again.

12. Tool Schemas

A tool definition has exactly three parts:

- **name** — short, machine-facing (e.g. `get_current_datetime`).
- **description** — free text describing what the tool does, *when* Claude should reach for it, and what it returns. This is **the single most important lever a developer has**, because it is the *only* documentation Claude has access to — there's no separate docstring, no comment, nothing external it can consult. A rule of thumb: 3–4 sentences, not a one-liner, explicit about units, defaults, output format, and the conditions under which the tool should be used. A vague description is the direct cause of two failure modes: wrong arguments, or not being called when it should be.
- **input_schema** — a JSON Schema object describing the arguments, with a `description` on *each individual property*, not just the top level. JSON Schema is a general-purpose, pre-existing data-validation spec — the API simply adopts it as the contract format for tool arguments.

Writing the underlying function

- Give the function and its parameters descriptive names.
- Validate inputs; throw on invalid ones rather than silently guessing.
- Give meaningful error messages — **Claude sees thrown error text** and can use it to retry with corrected arguments. This is the seed of the `is_error` mechanism (Chapter 15): a specific message ("dateFormat cannot be empty") is far more actionable to Claude than a generic one.

13. Claude's Decision to Use a Tool

Passing `tools` in a request **offers a capability, it doesn't force its use**. The default `tool_choice` is `{ type: 'auto' }` — Claude independently judges, per request, whether a tool is relevant, exactly as it judges whether any other sentence belongs in its answer. The same tool array passed alongside two different questions — one where the tool is relevant, one where it isn't — produces a `tool_use` response for the first and a plain `text / end_turn` response for the second.

`tool_choice` can override this default:

- `{ type: 'tool', name: '...' }` — force a specific tool to be called.
- `{ type: 'none' }` — disable tool calls entirely for this request.

Built-in server-side tools (Chapter 19) behave differently here: some (e.g. web search) **cannot** be forced this way at all — the API rejects a forced call outright, requiring auto-triggered calls only.

14. The Response Shape: Content Blocks and `stop_reason`

Once `tools` is part of a request, `response.content` can no longer be treated as "just a string" — it's an array whose block types depend on what Claude decided to do:

```
{ "type": "text", "text": "... " }
{ "type": "tool_use", "id": "toolu_01...", "name": "get_current_datetime", "input": {
  "date_format": "YYYY-MM-DD" } }
```

- `id` — uniquely identifies this specific call, later matched against a `tool_result` 's `tool_use_id` .
- `name` — the tool's name, exactly as declared in its schema.
- `input` — **already a parsed object**, not a JSON string — no `JSON.parse()` needed.

`response.stop_reason` is the primary dispatch signal, checked *before* inspecting content blocks:

- `"tool_use"` — at least one `tool_use` block is present; Claude is paused, waiting on a result.
- `"end_turn"` — Claude answered directly, no pending tool call.

Multi-block responses are normal, not an edge case. A single response can contain a `text` block *and* a `tool_use` block together — e.g. "Let me check the current time for

you," immediately followed by the tool call. Code that assumes "the whole answer is block 0," or hardcodes a block's index, is fragile. The robust pattern is to iterate all blocks and switch on `block.type`, and to locate a `tool_use` block with a filter/find on `type` rather than by position.

15. Sending Results Back: `tool_result`

Once your tool function has run, its output goes back to Claude as a `tool_result` content block — living inside a message with `role: 'user'`, not `'assistant'`. From the API's perspective this is "data coming back to Claude," conceptually the same as a person's follow-up message, even though a program produced it.

```
{ "type": "tool_result", "tool_use_id": "toolu_01...", "content": "2026-08-03",  
  "is_error": false }
```

- `tool_use_id` — must exactly match the originating `tool_use` block's `id`, so Claude can pair a specific result to a specific request (matters once multiple calls are in flight — Chapter 16).
- `content` — the tool's output, as a string.
- `is_error` — `true` signals the call failed, with `content` holding the error message. Paired with the "meaningful error messages" advice in Chapter 12, this is how Claude learns a call didn't succeed and can attempt a corrected retry — meaningless if the tool can never actually fail.

A common, API-breaking mistake: the follow-up request that sends the `tool_result` back must *still* include the same `tools` array, even if no further tool call is expected. Without it, Claude has no schema to interpret the `tool_use` / `tool_result` blocks already in history.

Conversation-history integrity: because Claude has no server-side memory, *every* prior turn — including raw `tool_use` blocks — must be replayed verbatim in `messages` on each subsequent call. When appending Claude's turn to history, push the **entire `response.content` array**, not just extracted text and not just the `tool_use` block in isolation: extracting only the text (or only the `tool_use` block, losing its `id`) silently breaks the next call, because the `tool_result` sent afterward has nothing correctly-shaped to attach to. After one round trip, history holds (at minimum) four entries: user's question → assistant's `tool_use` → user's `tool_result` → assistant's final text answer — all four must be replayed on any further turn.

The name↔function link is convention, not framework-enforced

A `tool_use` block's `name` is just a string — nothing in the SDK automatically calls real code when Claude "asks" for a tool by name. Dispatching that string to actual executable code is application logic, typically a **dispatch table**:

```
TOOL_FUNCTIONS = {  
  get_current_datetime: (input) => getCurrentDatetime(input.date_format),  
  ...  
}
```

This table is also the natural place to translate between wire-format field names (matching `input_schema`, often `snake_case`) and a language's own idiomatic parameter naming — the schema's property names and the function's real parameter names don't have to match syntactically as long as the dispatch entry bridges them.

16. Multiple Tools and the Agentic Loop

This unit is careful to separate two things that look similar but aren't.

(a) Multiple `tool_use` blocks in one response ("parallel" calls)

Claude can request more than one tool — or the same tool twice — within a single response (e.g. "what's 10+10 and what's 30+30?"). Handling this is a filter+map over `response.content` rather than a find-one-and-done:

```
toolResultBlocks = response.content  
  .filter(b => b.type === 'tool_use')  
  .map(runTool)
```

Each resulting `tool_result` carries its own `tool_use_id`, so several results can be returned together (as one `user` message containing multiple blocks) and Claude still correctly matches each to its originating request.

(b) A single question needing several tools in sequence

Distinct from (a): "What day is 103 days from today?" needs `get_current_datetime` first, then `add_duration_to_datetime` using that result — the second call can't happen until the first's result is known. The two calls happen across separate turns of a loop, not within one response, so this needs an actual loop, not a fixed two-step exchange.

The agentic loop as a state machine

```
while (true) {  
  response = callClaude(tools)
```

```

history.push({ role: 'assistant', content: response.content }) // FULL content

if (response.stop_reason !== 'tool_use') {
  return textFrom(response) // Claude is done
}

toolResultBlocks = response.content
  .filter(b => b.type === 'tool_use')
  .map(runTool)
history.push({ role: 'user', content: toolResultBlocks }) // feed results
back, loop again
}

```

The loop condition is `response.stop_reason : "tool_use"` means run whatever tool(s) this response requested and call Claude again; anything else means Claude has finished. This single shape handles both (a) and (b) at once — the filter/map inside one iteration covers multiple simultaneous calls, and the `while` wrapping multiple iterations covers multi-step chains; they compose rather than needing separate code paths.

Why extracting text on every iteration matters, not just the final one: a response can carry explanatory text *alongside* a `tool_use` block ("I'll find out what day it is 103 days from today.") — surfacing text only from the loop's last iteration would silently drop every intermediate remark Claude made while working through the chain, which is exactly the running commentary a real user would expect to see.

A worth-flagging pitfall: it's easy to mis-thread roles here — Claude's own reply must always be recorded as the `assistant` turn, never `user`. Recording two consecutive `user` turns with nothing in between breaks the strict-alternation rule from Chapter 1.

Once more than one tool exists, the dispatch table from Chapter 15 becomes necessary infrastructure, not an optional nicety — a single hardcoded call to one function only works while exactly one tool is available.

17. Multi-Turn Conversations With Tools

Distinguish "a loop that runs several tool calls to answer *one* user message" (Chapter 16) from an actual ongoing conversation, where a human alternates turns with Claude over an unbounded session, and *each individual turn* may itself trigger the internal tool loop before producing a reply.

Mechanically, this is the same tool loop wrapped in an outer read-input loop, sharing **one continuously growing history** across all turns rather than resetting it between questions. Because history persists, later turns can genuinely reference facts established by earlier tool calls still sitting in it — e.g. asking "what day is 103 days from

today?" and later, in the same session, "and 30 days after that?" resolves correctly only because the first exchange's tool results remain part of the replayed history. No new tool-handling mechanics are introduced at this stage — the dispatch table, the loop, and the block-handling all carry forward unchanged.

18. Streaming Tool Use

Regular streamed text arrives as steady, token-by-token deltas. When a streamed request also declares `tools`, a tool call's *arguments* can also stream incrementally via `input_json_delta` events — but not steadily.

Key gotcha — JSON arrives in validated bursts, not continuously: the API buffers everything belonging to a tool call's *current top-level key*, validates that key's value against the tool's `input_schema`, and only then releases the buffered chunk(s) for that key in one burst. Practically: silence, then a burst for one property (even a nested object), then silence, then a burst for the next property. A direct consequence: under this default mode, the JSON delivered at each step is always **valid, schema-checked partial JSON** — no `try/catch` needed around parsing it.

Detecting the start of a tool call is a gap in the high-level API — there's no dedicated convenience event that fires the instant a `tool_use` block begins; the natural "block complete" event only fires once all its JSON has already streamed through. To learn a tool's name as soon as Claude starts calling it, you have to drop to the raw event stream and check for a block-start event whose block type is `tool_use` — this is the one common case where bypassing the high-level convenience wrapper is necessary.

Fine-grained tool streaming (an opt-in, beta-gated mode) removes the buffering/validation delay entirely — JSON chunks arrive the instant they're generated. The tradeoff: since validation is skipped, the JSON is **no longer guaranteed valid or complete** at any given snapshot, so consuming code must wrap parsing in error handling. Choose this only when the default buffering delay is itself a real UX problem worth trading away guaranteed-valid parsing for.

19. Built-in Tools vs. Custom Tools

Custom tools (everything above) are ones where you write the full definition — `name`, `description`, `input_schema` — from scratch, and your own code executes the logic and returns a `tool_result`.

Built-in tools are Anthropic-provided; Claude already knows the complete schema and calling conventions internally. The declaration sent over the wire is a minimal stub identifying which tool and which version — no `input_schema` is sent at all, because Claude expands the stub into its full internal spec itself, e.g. `{ type: 'text_editor_20250728', name: 'str_replace_based_edit_tool' }`. Built-in tools split further into two execution models:

(a) Client-side built-in tools — e.g. the text editor tool

Claude knows the full schema and decides when/how to call standardized commands (`view`, `str_replace`, `create`, `insert`), but has **no filesystem of its own**. Every invocation still comes back as an ordinary `tool_use` block your own code must execute and answer with a `tool_result` — exactly like a fully custom tool. The only difference from a custom tool is that the schema/spec itself is predefined and versioned by Anthropic.

- **Versioning is a hard-coupled pair:** the `type` and `name` strings change together across model generations — mismatching them is a 400. Always confirm the current pairing when moving to a new model, since older documented pairs silently stop being valid on newer ones.
- **Capabilities can disappear between versions** — a command available on an older spec (e.g. an "undo" capability) can be removed entirely from a newer one; a backing implementation for a dropped command becomes dead code, not something to defensively keep.
- **Security stays entirely on the developer.** Any path-like argument arriving in a tool call is untrusted model output, exactly like user input — it must be sandboxed (resolved against a fixed base directory, rejected if it would resolve outside it), or a hallucinated/injected path (`../../etc/passwd` -style traversal) becomes a genuine vulnerability. The built-in spec does not provide this protection for you.

(b) Server-side built-in tools — e.g. the web search tool

A fundamentally different execution model: Anthropic's own infrastructure runs the tool logic and folds results directly into the response. **There's no `tool_use` block to act on and no result-sending loop to write at all** — the declaration in the request is the entire client-side implementation.

- **Org-level enablement gate** — some server-side tools must be turned on for the organization in account settings before use; declaring the tool while the org setting is off is a hard failure, not a silent no-op, which can look like a client bug when it's actually a missing configuration step.

- **A usage-limiting field caps searches, not results** — one search can return multiple hits, and Claude may deliberately run several searches before it's satisfied; the cap is a cost/runaway-usage guardrail sized per search invocation.
 - **Cannot be forced** — `tool_choice: { type: 'tool', name: '<built-in>' }` is rejected outright; the only lever to reliably provoke use is prompt phrasing that makes a live/specialized lookup clearly more valuable than the model's own training-data knowledge.
 - **Response block taxonomy**: a `text` block (narration/answer), a "server tool use" block (records the exact operation performed — e.g. the literal search query), and a "server tool result" block (the tool's output). **Success and failure are not distinguished by HTTP status** — both come back as a normal 200; success looks like an array of result blocks, failure looks like a single error-shaped object in the same field. Client code must check the shape, not the status code.
 - **Tool-version evolution can change block topology, not just the version string** — a newer "dynamic filtering" capability lets Claude write and run a small script that calls the search capability programmatically and filters raw results itself, producing a *tree* of nested "server tool use" blocks (a filtering script with nested searches pointing back to it via a "caller" reference) instead of one flat search→result pair. This can also make citations (Chapter 30) less reliable, since citation matching depends on an exact substring match between Claude's text and the raw result content — once results pass through Claude's own filtering script, that exact correspondence can break. Treat citations as an optional enhancement rendered only when present, and build any authoritative "sources used" list from the raw result blocks themselves.
-

Part IV — Retrieval-Augmented Generation (RAG)

20. Why RAG Exists

Some documents are too large to fit in a single prompt (the running example across this unit: an 800-page financial-style document).

- **Naive option — stuff the whole thing in.** Fails against hard prompt-length limits, degrades effectiveness even when it technically fits (very long prompts

dilute the model's focus), and costs more and runs slower (you pay per token, relevant or not).

- **RAG (Retrieval-Augmented Generation) option** — preprocess the document into pieces (`chunks`), then at query time search for and inject only the relevant chunk(s):

```
Naive: <financial_document>{ full 800-page doc }</financial_document>
RAG:   <report>{ one retrieved chunk }</report>
```

Benefits: Claude focuses only on relevant content, it scales to documents (or whole corpora) that could never fit in one prompt, it works across multiple documents at once, and prompts stay cheap and fast. Costs: you now need a chunking step and a retrieval mechanism, and a retrieved chunk is an **imperfect proxy** — it may lack context Claude needs, and there's no single best chunking approach; it depends on the document type.

When to reach for RAG: it's worth the added complexity when documents are very large, span multiple sources, or when cost/latency must be optimized — a trade of simplicity for scalability and efficiency.

21. Chunking Strategies

Chunking quality directly determines whether RAG works. Bad chunking → irrelevant context retrieved → confident wrong answers.

A recurring cautionary scenario across this whole unit: a corpus with both a medical-research section and a software-engineering section. The query "How many bugs did engineers fix this year?" risks surfacing the medical section, because it literally contains the word "bug" (the insect/ contaminant sense), even though the software sense was intended. This exact scenario resurfaces in the embeddings chapter (proving semantic search fixes it) and the lexical-search chapter (showing where lexical alone would still struggle).

Strategy	Method	Strength	Weakness
Size-based	Fixed-length character windows	Simple, works on <i>any</i> document type	Cuts words/sentences mid-thought, loses cross-chunk context

Size-based + overlap	Same, but each chunk repeats a tail slice of the previous one	Fixes boundary-splitting — a sentence split at a boundary still appears intact somewhere	Still arbitrary; doesn't respect meaning or structure
Structure-based	Split along the document's own structure (e.g. Markdown headers)	Cleanest, most meaningful chunks — each is a complete logical section	Only works if structure is reliable (fails on plain text, arbitrary PDFs)
Sentence-based	Split into sentences, group N per chunk with sentence-level overlap	Practical middle ground for plain-text documents	Less semantically coherent than structure- or true semantic-based chunking
Semantic-based	Split into sentences, then use a model to measure relatedness between consecutive sentences and group related ones	Most relevant/coherent chunks possible	Computationally expensive, most complex to build

No universal best. Structure-based wins when formatting is guaranteed (e.g. internally authored reports with a fixed template); sentence-based is a solid default for plain text; size-based-with-overlap is the most reliable fallback — works on literally anything, including code — and is common in production precisely because it's simple and robust, not because it's the most accurate.

The general size tradeoff: too-small chunks lose surrounding context (a fact or reference gets orphaned from what it modifies); too-large chunks dilute the relevance signal and waste tokens once retrieved — you're back to paying for irrelevant text, just less of it. Overlap specifically mitigates the "too small, cut mid-thought" failure.

22. Embeddings and Semantic Search

Once chunks exist, finding the relevant ones is a search problem. `Keyword search` matches exact literal words; `semantic search` compares *meaning* via `embeddings`, matching what a chunk is *about* regardless of literal word choice.

What a text embedding is, conceptually:

1. Feed text into an embedding model.
2. It outputs a long vector — a list of numbers, typically ranging -1 to +1 per dimension.
3. Collectively, the numbers encode features of the text's meaning.

Dimensions are not individually interpretable. It's tempting to imagine one dimension means "happiness" or another means "ocean-relatedness," but what each dimension captures is learned during training and isn't human-readable. The vector is used as a *whole*, via similarity comparisons, never by inspecting individual components.

Embeddings typically require a dedicated embedding model/service, separate from the LLM itself — a reminder that an LLM provider and an embeddings provider can be different systems in a RAG stack.

inputType : query vs. document — a subtle but important asymmetry. Embedding models can weight "the thing being searched for" differently from "the thing being indexed," to make the eventual similarity comparison more accurate. Mismatching these (e.g. indexing chunks in query mode) silently degrades search quality without erroring — a hard-to-detect bug class in RAG systems generally.

An embedding by itself answers nothing — the mechanism that actually does "search" is *comparing* embeddings (via `cosine similarity`) to find the chunk vector closest to the query vector, covered next.

23. The Full RAG Pipeline, Worked Numerically

A toy 2-dimensional walkthrough, deliberately reusing the medical/software "bug" scenario so literal keyword overlap would mislead a naive approach:

1. **Chunk** the source text (e.g. structure-based, Chapter 21).
2. **Generate embeddings** for each chunk. Imagine (purely for illustration) a 2-D embedding where dimension 1 = "how much this text is about medicine" and

dimension 2 = "how much about software" — real embeddings (e.g. 1024-D) work the same way, just with dimensions that aren't individually readable:

- Medical chunk → `[0.97, 0.34]` (the word "bug" nudges the software dimension up slightly)
- Software chunk → `[0.30, 0.97]` (the phrase "infection vectors" nudges the medical dimension up slightly)

3. **Normalize** — embedding APIs automatically scale every vector to magnitude 1.0 (unit vectors). Geometrically: points on a unit circle (2-D) or unit hypersphere (high-D) — what matters for comparison is *direction*, not magnitude.
 - Normalized: Medical `[0.944, 0.331]` , Software `[0.295, 0.955]`
4. **Store in a vector database** — "a database built for storing and searching long lists of numbers efficiently." Steps 1–3 are **preprocessing**, done once, ahead of any user query.
5. **Embed the user's query**, using `inputType: 'query'` (as opposed to `'document'` for indexed chunks — same model, different mode, because the two play different roles in the comparison). Example query embeds to `[0.1, 0.89]` → normalized `[0.112, 0.993]` — visually close to the Software chunk's shape.
6. **Rank by cosine similarity** — the cosine of the angle between two vectors, ranging **-1 to 1**: close to **1** means highly similar (same direction), close to **0** means unrelated (perpendicular), close to **-1** means opposite. Worked result: query-Software = **0.983** (very high) vs. query-Medical = **0.398** (much lower) — correctly retrieving the Software chunk despite the Medical chunk literally containing "bug." Cosine distance is `1 - cosine similarity` , flipping the scale so 0 = most similar (a convention many vector-DB tools use).
7. **Build the final prompt** — the same template shape as the naive approach in Chapter 20, but `<report>` now holds only the one winning chunk.

This is the mechanism that concretely prevents the "bug" failure mode threatened back in Chapter 21 — not just a cautionary tale, but something the math actually resolves.

24. Vector Indexes

Mapping the walkthrough above onto a real pipeline gives five operational steps: chunk text, generate embeddings, create a vector store and add each embedding, embed the user's question, and search the store for the most relevant chunk(s). Steps 1–3 are preprocessing (once); steps 4–5 happen per query.

- **Batch embedding generation.** Generating embeddings for many chunks in one batched call, rather than one call per chunk, avoids rate-limiting and is standard practice.
- **Store the original text alongside its embedding, not just the vector.** The embedding is only useful for *finding* the right chunk — once found, you need the actual source text to hand to the model. This "document + vector together" pairing is a core structural feature of a vector index.
- **The conceptual `VectorIndex` interface:** a constructor takes a distance metric (`cosine` or `euclidean`) and an embedding function; `addVector` stores a precomputed vector + its document (enforcing consistent dimensionality across the store); `addDocument(s)` embeds text and stores it; `search(query, k)` accepts a raw string (auto-embedded) or a precomputed vector, returning the `k` nearest `[document, distance]` pairs, nearest first.
- **Two distance metrics:** `cosine distance` (`1 - cosine similarity` , direction-based) vs. `euclidean distance` (straight-line distance between vector endpoints) — different notions of "closeness."
- **Retrieval is probabilistic, not deterministic ground truth** — real embedding models produce results consistent with, but not numerically identical to, toy examples; a different model snapshot can shift the exact scores or ranking of near-ties.

A one-line definition worth keeping: RAG is about turning text into numbers, storing those numbers so they can be searched efficiently, and using mathematical similarity — not exact word matching — to decide what's relevant to a question.

This basic version works for a lot of cases, but not all — semantic search has a specific blind spot, covered next.

25. Lexical Search and BM25

Semantic search is strong at meaning/paraphrase (it can find "leg exercises" content for a "quad workouts" query with zero literal word overlap), but weak at **exact-token lookups** — searching for a specific identifier like `INC-2023-Q4-011` can retrieve a conceptually-adjacent section that never contains that literal ID, while under-ranking the section that does, because semantic search matches concepts, not literal strings, and gives no special weight to an exact match.

	Semantic search	Lexical search
--	-----------------	----------------

Compares	Embeddings (meaning vectors)	Literal tokens (actual words/terms)
Good at	Conceptual/paraphrased matches	Exact terms: IDs, error codes, names, specific phrases
Blind spot	No special respect for exact string matches	No understanding of synonyms/paraphrasing
Mechanism	Embedding model + cosine distance	Token overlap + frequency weighting (BM25)

Neither approach dominates — they fail in complementary ways, which is what motivates **hybrid search** (Chapter 26): run both in parallel and merge results, gaining semantic reach and lexical precision simultaneously.

BM25 ("Best Match 25")

The standard lexical-search scoring algorithm, in four conceptual steps:

1. **Tokenize the query** into individual terms.
2. **Document frequency** — for each term, count how many documents in the corpus contain it *at all* (presence, not count).
3. **IDF (inverse document frequency)** — weight by rarity: a term appearing in almost every document ("the") contributes almost nothing to relevance; a term appearing in only one or two documents (a rare ID) is a strong, specific signal.
4. **Score and rank** — for each document, sum (term frequency in doc) × (term's IDF weight) across all query terms, with two correction factors:
 - **k1** (term-frequency saturation, default 1.5) — caps how much each *additional* occurrence of a term keeps adding to the score, so a document repeating a term 50 times doesn't dominate regardless of true relevance.
 - **b** (document-length normalization, default 0.75, range 0–1) — corrects for long documents scoring higher purely by containing more words overall. **b=0** ignores length entirely; **b=1** fully normalizes.

This is pure term statistics — no meaning or semantics involved at all.

Tokenization nuance: real tokenizers are typically mechanical — lowercase everything, split on runs of non-word characters — so an ID like `INC-2023-Q4-011` fragments into pieces (`inc` , `2023` , `q4` , `011`). BM25 can still work because the fragments are

collectively still rare in an unrelated corpus, but this is a real limitation: lexical search is only as good as its tokenization, and naive tokenizers aren't ID-aware.

Score direction: raw BM25 scores are "higher = more relevant" — the opposite convention from cosine *distance*, where "lower = more similar." (Chapter 26 revisits this — whether it actually matters for merging turns out to be more subtle than it first appears.)

26. Hybrid Search and Reciprocal Rank Fusion

Semantic search and lexical search each produce their own ranked result list, but their **scores live on incomparable scales** — a cosine distance of `0.234` and a BM25 score of `0.184` measure fundamentally different things. There's no principled way to average or directly compare raw scores across engines.

A common interface is a structural precondition for merging: any index usable in a hybrid retriever needs to expose the same shape — `add document(s)`, and `search(query, k)` returning ranked `[document, score]` pairs. The merge mechanism doesn't care *how* an index computes relevance internally, only that it can produce a ranked list.

Reciprocal Rank Fusion (RRF)

The standard hybrid-merge technique. Core idea: **sidestep the incomparable-scores problem entirely by discarding scores and working with rank position only.**

1. Run the query against each index independently; get each index's own ranked list, each on its own score scale.
2. Convert scores to **ranks** (1st, 2nd, 3rd...) within each list, then combine into one table per document showing its rank in each index — now every item has one rank per index, all on the same comparable ordinal scale regardless of the underlying scoring math.
3. Apply the RRF formula:

$$\text{RRF_score}(d) = \sum \text{over indexes } i \text{ of } 1 / (k + \text{rank}_i(d))$$

`k` is a smoothing constant controlling how much a rank difference matters: larger `k` flattens the curve (rank 1 vs. rank 2 barely differ), smaller `k` makes top ranks count much more heavily. `k = 60` is the standard default from the original RRF paper.

4. **Sort by RRF score descending** — note the direction flips relative to the raw inputs (raw distances/BM25 scores are "lower is better"; the final RRF score is "higher is better").

A common misconception worth preempting: it's tempting to think BM25's score direction needs to be normalized to match cosine distance before you can "merge and re-rank" the two result sets. This would matter for a naive "average the two scores" strategy — but **RRF never looks at score values at all**, only rank position, so the normalization concern is irrelevant to RRF specifically.

Deduplication: when the same document is added to multiple indexes, results need deduplication by document identity (not by re-comparing content) before or while merging.

Why hybrid search wins in practice: querying for a specific incident ID against a combined semantic+lexical retriever can produce a *tie* between two sections that each engine ranks differently — lexical favoring the section with more literal ID mentions, semantic favoring the section more conceptually "about" incident response — each earning one 1st-place and one 2nd-place finish, landing on an identical fused score. This is the canonical illustration of hybrid search's value: **both signals agreeing two results are strong, even while disagreeing on exact order**, while a merely-generic-vocabulary section correctly stays far behind in the fused ranking. The mechanism generalizes beyond exactly two indexes — a retriever can fan a query out to any number of underlying indexes and reconcile arbitrarily many ranked lists the same way.

Part V — Advanced Capabilities

27. Extended Thinking

What it is: Claude gets a visible scratchpad — a `thinking` content block — to reason in before producing its final `text` answer. This is not hidden internal computation; it's returned in the response structure alongside the answer.

Cost/latency tradeoff: thinking tokens are billed and add latency. The recommended workflow is to build and tune the prompt first, measure baseline accuracy *without* thinking, and only enable it if that's insufficient — it's not a default-on feature.

Adaptive thinking vs. a fixed budget: older models used `thinking: { type: 'enabled', budget: N }` — a fixed token allowance the developer set upfront. Current models reject this and instead use `thinking: { type: 'adaptive' }`, where Claude

decides for itself, per request, how much to think. The depth/cost dial moves to a separate parameter, `output_config: { effort: 'low' | 'medium' | 'high' | 'xhigh' | 'max' } .`

display controls visibility, not occurrence. Even with adaptive thinking and no `display` set, a `thinking` block is still returned but with an empty `.thinking` string — thinking still happens and is still billed, it's just not surfaced. Setting `display: 'summarized'` returns a human-readable recap of the reasoning instead of nothing.

signature : every `thinking` block carries a signature — a cryptographic integrity token meant to prevent a developer from hand-editing Claude's prior reasoning and feeding a tampered version back in a later turn (a potential steering/safety bypass vector). The practical rule: never reconstruct or hand-edit a thinking block — pass response content back unchanged.

Redacted thinking: when Claude's safety systems flag its own reasoning as sensitive, the block type becomes `redacted_thinking` with an encrypted `data` field instead of plain text — code must be prepared to receive and pass this through without crashing, even though it can't be read.

Known incompatibilities: response prefilling (Chapter 4) is incompatible with extended thinking, and `temperature` is documented as generally incompatible with it too.

Interleaved thinking with tool use: with adaptive thinking, Claude can think again *between* tool calls within the same turn, automatically, with no separate opt-in. Practical implication: an agentic loop mixing thinking and tools must append **all** returned content blocks — both `thinking` and `tool_use` — back into history, not just the tool-use ones, since they can be interleaved in `response.content` .

When to use it: nontrivial, multi-step, or agentic tasks where a single-pass answer is unreliable — not simple/cheap queries.

28. Vision (Image Analysis)

Message structure: an image is just another **content block** inside a user message's content array, alongside `text` blocks — no separate endpoint or request shape for vision.

- `{ type: 'base64', media_type, data }` — you read and base64-encode the file yourself; `media_type` must accurately match the format (`image/jpeg` , `image/png` , `image/gif` , `image/webp`).

- `{ type: 'url', url }` — Claude fetches the image server-side.

Claude's answer comes back as an ordinary `text` block — vision doesn't change anything else about history, system prompts, or tool use.

Limits worth knowing as cost/design constraints:

Limit	Value
Images per request	up to 100 (across the whole request)
Max size per image	5MB
Max dimension, single image	8000px
Max dimension, multiple images sharing a request	2000px (further downsampled to fit)
Approx. token cost	$(\text{width_px} \times \text{height_px}) / 750$

Large images aren't "free" just because they count as one attachment — resolution directly drives cost.

Ordering matters. Claude processes images in the order they appear in the content array. If a prompt needs to reference "the second image," the descriptive text block should come *after* the image blocks, not before.

Prompting techniques specific to vision (the same general prompt-engineering principles as text — Chapter 9 — but they matter more here, since direct questions on images are more error-prone than on text):

- **Step-by-step / explicit methodology** — rather than a direct "how many X are in this image?", instruct Claude to work through a numbered procedure (identify and number each item individually, then independently re-verify with a different counting method). Forcing an independent cross-check catches errors a single pass misses.
- **One-shot / multi-shot with images** — include a reference image paired with a known-correct answer before the real target image, the visual analog of text few-shot examples, to anchor Claude's expected level of care and output format.
- **Structured checklists for auditable judgments** — for compound judgment tasks (e.g. a risk assessment from a satellite image), break the ultimate

question into an explicit sequence of sub-assessments (locate the subject, evaluate specific risk factors one at a time, quantify each) and only derive the final score after all sub-steps, each justified. A bare "give me a score" prompt skips the reasoning that makes the output trustworthy and auditable.

29. PDF Processing

Message structure: a PDF is sent as a `document` content block (not `image`), alongside a `text` block in the same content array — structurally identical to the image pattern, just a different block type and `media_type` (`application/pdf`), with the same base64-encoding approach. Everything else about the request/response flow and multi-turn conventions is unchanged.

What Claude actually "sees" is the key conceptual point: it processes a PDF *holistically*, not merely as extracted raw text. It reads body text, embedded images and charts, tables (including relationships between the data within them), and overall document structure/formatting.

Why this matters: a single request can combine what would otherwise require separate text-extraction and image-analysis passes — e.g. answering a question that connects a chart on one page with a table on another, or producing a holistic summary that accounts for layout, not just extracted text.

30. Citations

Why they matter: without citations, a document-grounded answer is a black box — no way to verify whether a claim reflects the actual source or the model's own priors. Citations create a traceable link from each generated claim back to the exact source passage that supports it, central to trust and verifiability of document-grounded output.

How it's enabled: two additional fields on the `document` block from Chapter 29:

- `title` — a human-readable name Claude uses to refer to the document in citation metadata.
- `citations: { enabled: true }` — turns on citation tracking for that document.

What changes in the response: with citations off, output is flat text. With citations on, *each* `text` block carries its own `citations` array — empty for sentences not grounded in the source, populated for sentences that are. Grounding is tracked at per-sentence granularity, not once for the whole response.

Two location schemes, determined by the *source* format, not a separate setting:

	Page-based (<code>page_location</code>)	Character-based (<code>char_location</code>)
Used when source is	a PDF	plain text
Location fields	<code>start_page_number</code> / <code>end_page_number</code> (end exclusive)	<code>start_char_index</code> / <code>end_char_index</code>

Both types also carry `cited_text` (the actual quoted passage), `document_index`, and `document_title` — a citation is self-describing: which document, which title, the exact quoted span, and where it sits in the source. The `document` block shape is otherwise identical regardless of source type — only `source.type` and the resulting citation's location flavor differ.

This naturally suits a "hover/click to see provenance" UI, surfacing the cited passage and location on demand next to the sentence it supports.

31. Prompt Caching

Why it exists: every request incurs preprocessing cost (tokenizing, context analysis) before generation even starts, normally discarded after the response is sent. If a large portion of the prompt prefix repeats across calls — a system prompt, tool schemas, a long document, a growing conversation history — caching persists that preprocessed state so a later request can read it back instead of redoing the work, yielding **lower latency and reduced cost** (cached tokens are billed at a fraction of the normal input rate).

Caching is opt-in, via cache breakpoints. A developer explicitly marks where the cacheable prefix ends by attaching `cache_control: { type: 'ephemeral' }` to a specific content block. Everything up to and including that block is treated as the cached prefix; anything after is processed normally. This attribute can be attached to blocks in the system prompt, tool definitions, or message content — anywhere a content block can appear. One practical consequence: a system prompt must use the verbose array-of- content-block form rather than a bare string, since the plain-string shorthand has no field to attach a breakpoint to.

Rules governing hits and misses:

- **Exact-match requirement** — the cache only hits if the content through the breakpoint is byte-for-byte identical to a prior request. Any upstream change, even one added word, invalidates the match — no partial credit. A miss isn't an error, just a fresh/cold write.
- **1024-token minimum** — content shorter than this is never cached regardless of a breakpoint, and the minimum applies to the *cumulative* content up to the breakpoint, not any single block alone.
- **1-hour TTL** — a cache entry expires if it isn't read again within an hour of its last write/read.
- **Fixed wire order** — requests serialize as tools, then system prompt, then messages, and breakpoints are evaluated in that order.
- **Up to 4 breakpoints per request** — enough to cache tools, a system prompt, and still have room to mark progressive points within a growing conversation.

Observability: `response.usage` reports `cache_creation_input_tokens` (tokens freshly written), `cache_read_input_tokens` (tokens read from an existing entry), and `input_tokens` (processed normally, outside any cached region). A single request with multiple breakpoints can show both creation and read fields nonzero simultaneously — some older content hit from cache, some newer content freshly written.

Cross-message caching: a breakpoint isn't pinned to one fixed message — placing it on the last content block of the *most recent* message caches everything before it too, across however many prior turns exist. In a multi-turn scenario, advancing the breakpoint forward each turn lets the cached prefix grow to cover the whole conversation-so-far incrementally.

This matters most for system prompts (sent unchanged every call), tool schemas (static across calls), and any long or repeated document or growing multi-turn history — the benefit scales with how much of the prompt prefix repeats, and how often.

32. Code Execution

What it is: a server-side tool (same family as web search — Chapter 19) that lets Claude run Python code inside an isolated sandboxed container as part of answering a request. Like other server-side tools, it's declared purely as a tool definition — no client-side execution loop needs to be written; the *server* runs the code.

Container characteristics:

- Isolated, with **no network access** — the Files API (Chapter 33) is the *only* channel for getting data in or generated output out.

- Claude can invoke code execution **multiple times within a single response**, iterating on its own intermediate output — running exploratory code first, inspecting the result, then producing a final analysis based on what it learned.
- Comes pre-loaded with common data-science libraries (pandas, numpy, matplotlib, etc.) — no environment setup needed in the prompt.

Response structure — the trickiest part: declaring the single `code_execution` tool grants access to *multiple* underlying sub-tools, and it's those specific sub-tool names that appear in response blocks — not a generic "code_execution" block type. Blocks include plain `text`, specialized `server_tool_use` blocks (e.g. a shell-execution sub-tool carrying the command run, or a file-operation sub-tool carrying file writes), and matching `*_tool_result` blocks. A shell-result block's content is effectively a discriminated union: an error variant (carrying an error code), or a success variant carrying stdout, stderr, a return code, and any files the command produced — those come back as **file IDs**, retrieved via the Files API, not inline bytes.

A practical pitfall worth flagging: don't trust type definitions or docs alone to predict exact block-type names returned by the real API — actual runtime responses can diverge from what generic/older references suggest. The reliable approach is to log the actual block types from a real response and match code against that.

Container reuse across requests: the response includes container identity/expiry info; passing that container's ID into a later request resumes the *same* container (same installed state, same files already on disk) instead of provisioning a fresh one each call — relevant for multi-turn analysis sessions that build on earlier work.

When useful: data analysis, calculations, and any task better solved by Claude actually executing verifiable code than by reasoning or guessing at a numeric or data-processing answer in free text.

33. The Files API

Core idea: an alternative to inlining base64 data directly in every request (as vision and PDF blocks do by default) — upload a file once, receive a stable file ID back, then reference that ID from any number of subsequent requests instead of re-sending and re-encoding the raw bytes each time.

Why it's worth using over inline base64:

- **Reuse across many requests** without re-uploading each time.
- **Efficiency for large files** — inlining big files as base64 bloats request bodies; referencing an ID keeps requests small regardless of the underlying file size.

- **The only data bridge for code execution containers** — since those containers have no network access, uploading input files and downloading generated output files via the Files API is the *sole* mechanism to move data in or out, in either direction.

Referencing an uploaded file differs by feature. For code execution, an uploaded file is attached via a container-oriented content block carrying the file ID, alongside a normal text instruction about what to do with it. Outside of code execution — referencing a file directly as an image or document source — uses a different block shape (a `source` object with a file-reference type). The two features consume file IDs through different content-block conventions.

Downloading generated files: when code execution produces new output (a plot, a processed dataset), it's not returned as inline bytes in the response — it comes back as a file ID that must be separately downloaded.

Security consideration: metadata about an uploaded or generated file (e.g. its filename) is untrusted input — a crafted response could supply a path-traversal-style filename. Any filename coming back from the API should be sanitized (stripped to just the base filename) before being joined onto a local output path.

Part VI — Agents and Workflows

Everything in Parts I–V is a building block: a request/response shape, a way to measure prompt quality, a way to let Claude act on the world, a way to ground it in documents too large to fit in context, and a set of special-purpose capabilities. This final part is about *composition* — how you assemble those blocks into something that reliably solves a real, possibly multi-step, user task.

34. Workflows vs. Agents

Two strategies for handling tasks that can't be completed by Claude in a single request. You've actually been building both throughout this book: any time you gave Claude tools and let it figure out how to use them (Part III), that was an agent. Any time you built a fixed multi-step pipeline (dataset generation → run → grade in Part II, or chunk → embed → retrieve in Part IV), that was a workflow.

The decision comes down to how well you understand the task ahead of time:

- **Workflows** — a series of calls to Claude meant to solve a specific problem through a predetermined series of steps. Use these when you can picture the exact flow Claude should go through, or when your app's UX constrains users to a set of known tasks.
- **Agents** — Claude is given a goal and a set of tools, and is expected to figure out how to complete the goal through them. Use these when you're not sure exactly what task or task parameters will be given to Claude.

Example: a web app where a user drops in an image of a metal part and the app produces a STEP file (a 3D model format) from it. Since there's a clear, predefined series of steps to go from image to model, this is a good workflow candidate:

1. Feed the image to Claude, asking it to describe the object.
2. Based on the description, ask Claude to model the object with a CAD library.
3. Create a rendering of the model.
4. Ask Claude to grade the rendering against the original image, fixing issues if any.

That last step is itself an instance of a named pattern, covered next.

35. Workflow Pattern: Evaluator-Optimizer

- **Producer** — takes input and creates output (in the example above, Claude modeling the part and rendering it).
- **Grader** — evaluates the output against some criteria.
- **Feedback loop** — if the grader doesn't accept the output, feedback goes back to the producer for improvement.
- **Iteration** — the cycle repeats until the grader accepts the output.

This is conceptually the same evaluate-then-improve loop from Part II (Chapter 9) — a general-purpose recipe that shows up wherever quality can be checked automatically and fed back as a correction signal. Identifying a pattern like this doesn't do anything by itself — you still have to write the code that implements it — but having a name for it turns "trial and error" into a repeatable recipe you can reach for deliberately.

36. Workflow Pattern: Parallelization

Some tasks look simple on the surface but become hard to get right in a single prompt. Parallelization breaks a complex task into several focused, independent pieces that run at the same time.

The problem: a material-designer app where a user uploads an image of a part and the app recommends the best material (metal, polymer, ceramic, composite, elastomer, or wood). A single simple prompt asking Claude to choose isn't reliable without specific per-material criteria — but cramming detailed criteria for every material into one massive prompt just makes Claude juggle many considerations at once, leading to worse results.

The pattern:

1. Send the same image to Claude multiple times simultaneously — once per material — each with its own specialized criteria.
2. Claude evaluates suitability for each material independently.
3. Collect all the individual results.
4. Send all results back to Claude in a final aggregation step, asking it to compare them and make a final recommendation.

```
metal criteria    ┌
polymer criteria ┌┐
ceramic criteria ┌┐→ Claude (×N, in parallel) → analysis results → Claude
(aggregator) → final recommendation
composite criteria└
```

The parallelized sub-tasks don't need to be identical — each can have its own prompt, tools, or criteria.

Benefits: focused attention (Claude concentrates on one aspect rather than balancing competing considerations), easier optimization (each sub-task's prompt can be improved independently), better scalability (adding a material is just another parallel request), and improved reliability (less cognitive load per call, more consistent results).

When to use it: a complex decision that breaks down into independent evaluations — multiple criteria, several options to compare, or decisions spanning different domains of expertise — where each sub-task can operate independently and contribute a distinct piece of analysis to the final decision.

37. Workflow Pattern: Chaining

A chaining workflow breaks a large, complex task into smaller, *sequential* subtasks that build on each other — as opposed to parallelization, where sub-tasks are independent and run at the same time.

Example: a social media tool that creates and posts videos automatically could chain, instead of one massive prompt:

1. Find related trending topics on Twitter.
2. Select the most interesting topic (Claude).
3. Research the topic (Claude).
4. Write a script for a short-format video (Claude).
5. Use an AI avatar and text-to-speech to create the video.
6. Post it to social media.

Only some steps need Claude — chaining is also the natural place to mix in non-LLM processing between steps (a Twitter API call, avatar/TTS rendering, posting).

Why chain instead of one big prompt: giving Claude one specific task at a time keeps it focused on doing that task well, rather than juggling multiple requirements at once. It splits a large task into smaller, non-parallelizable subtasks (each depends on the previous one's output), allows non-LLM processing between tasks, and keeps Claude focused on one aspect at a time.

The long-prompt problem, and chaining as its fix: asking Claude to write a technical article that must simultaneously not mention it's AI-written, avoid emojis, skip casual language, and stay professional — even with all constraints clearly stated — can still produce output violating some of them. A two-step chain fixes this:

1. **Generation step** — send the initial prompt, accepting the first result may not be perfect.
2. **Revision step** — send a focused follow-up with the article Claude just wrote and targeted revision instructions:

```
Revise the article provided below. Follow these steps to rewrite the article:  
1. Identify any location where the text identifies the author as an AI and  
   remove them  
2. Find and remove all emojis  
3. Locate any cringey writing and replace it with text that would be written by  
   a technical writer
```

This works because Claude can focus entirely on revision rather than balancing content creation with constraint adherence simultaneously.

When to use chaining: complex tasks with multiple requirements, Claude consistently ignoring some constraints in long prompts, a need to process or validate outputs between steps, or wanting each interaction to stay focused and manageable.

38. Workflow Pattern: Routing

Routing solves the problem of different request types needing different handling: instead of one generic prompt for every case, categorize the incoming request first, then send it to a specialized pipeline for that category.

The problem: a video-script generator can't use one script-writing prompt for every topic — "programming" calls for educational content with clear explanations, while "surfing" works better as entertainment-focused content emphasizing excitement and visual appeal.

Setting up categories, each with its own specialized prompt template — e.g. Entertainment, Educational, Comedy, Personal vlog, Reviews, Storytelling. The educational template might ask Claude to "develop a clear, engaging script that transforms complex information into digestible insights using relatable examples and thought-provoking questions."

Two steps in practice:

1. **Categorization** — send the topic to Claude, asking it to categorize into one of the predefined genres:

```
Categorize the topic of a video into one of the listed categories:
<topic>Python functions</topic>
<categories>
- Educational
- Entertainment
- Comedy
- Personal vlog
- Reviews
- Storytelling
</categories>
```

2. **Specialized processing** — use the category result ("Educational") to pick the matching prompt template and generate the actual content.

```
User Input → Router (Claude call) → one of:
    |─ Entertainment pipeline
    |─ Educational pipeline
    |─ Comedy pipeline
    └─ ...
(user input only ever reaches ONE branch)
```

The key insight: user input only goes to **one** specialized pipeline, not all of them — letting each pipeline be highly optimized for its specific use case.

When to use routing: the app handles diverse request types needing different approaches, categories can be clearly defined, categorization can be handled reliably by Claude, and the benefit of specialized processing outweighs the overhead of the extra routing step. Especially valuable for customer service bots and content generation tools.

39. Agents and Tools

Agents are a shift away from the structured workflows above. Workflows are great when you know the exact steps needed; agents shine when you don't. You give Claude a goal and a set of tools (Part III), and let it figure out how to combine them to achieve the objective — Claude formulates its own plan, stated or unstated.

The goal is to reliably and economically complete tasks — often harder to achieve with agents than with workflows, since you're trading control for flexibility. The benefit: agent flexibility allows a more flexible UX — you build the agent once, verify it works reasonably well, and it can then handle a wide range of problems you didn't explicitly plan for.

Tools make the agent. The real power comes from combining simple tools in unexpected ways. Take a basic set of datetime tools — `get_current_datetime`, `add_duration_to_datetime`, `set_reminder` — individually simple, but Claude can chain them for much more complex requests: "What's the time?" needs just the first; "What day of the week is it in 11 days?" chains the first two; "Remind me to go to the gym next Wednesday" potentially uses all three in sequence. Claude can also recognize when it's missing information — asked "When does my 90-day warranty expire?", it knows to ask when the item was purchased first.

Tools should be abstract, not hyper-specialized. Claude Code is the canonical example: it has generic tools — `bash` (run any command), `read`, `write`, `edit`, `glob` (find files), `grep` (search file contents) — and notably *no* specialized tools like "refactor code" or "install dependencies." Claude figures out how to compose the generic tools to accomplish these tasks instead, letting it handle countless scenarios the developers never explicitly planned for.

Best practice: combinable tools. A social media video agent might get `bash` (FFMPEG access), `generate_image`, `text_to_speech`, and `post_media`. This supports both simple workflows (create and post a video) and more interactive experiences — the agent generates a sample image first, gets user approval, then proceeds with video creation, adapting its approach based on feedback — something difficult to achieve with a rigid workflow.

40. Environment Inspection

Claude operates blind by default — it needs a way to observe and understand the results of its own actions to work effectively as an agent.

Why it matters: with computer use, every action Claude takes (typing, clicking) is followed by a screenshot so it can see what happened. This isn't optional — a click could navigate to a new page, open a menu, or do something else entirely. Without seeing the result, Claude has no way to know whether the action succeeded or what the environment's new state looks like. The same principle applies to file edits: before modifying a file, Claude should read its current contents first, or it risks breaking existing functionality it never actually saw.

Guiding inspection via system prompts. For complex tasks, system prompt instructions can steer Claude toward inspecting its own output. E.g. a video generation agent's system prompt might say, after generating videos: use `bash` to run `whisper.cpp` and generate a caption file with timestamps to verify dialogue placement; use `bash` to run `FFmpeg` and extract a screenshot from every second of the video to verify it looks as expected.

Benefits: better progress tracking (Claude can gauge how close it is to done), error handling (unexpected results can be detected and corrected), quality assurance (output verified before the task is considered complete), and adaptive behavior (Claude adjusts based on what it observes).

Practical implementation: when designing an agent, always ask "How will Claude know if this action worked?" — reading file contents before modifications, taking screenshots after UI interactions, checking API responses for expected data, validating generated content against requirements. Environment inspection is what turns Claude from a blind executor of commands into an agent that can understand and adapt to its working environment.

41. Choosing Between Workflows and Agents

	Workflows	Agents
Summary	A predefined series of calls to Claude meant to solve a known problem or set of problems.	Claude is given a set of basic tools and is expected to formulate a plan to use them to complete a task.

	Used when you can picture the flow of steps ahead of time.	
Benefits	Claude can often focus on one subtask at a time, generally leading to higher accuracy. Far easier to evaluate and test, since you know each exact step.	Allows for a more flexible UX. Far more flexible task completion — Claude can combine tools in unexpected ways to complete a wide variety of tasks.
Downsides	Far less flexible — dedicated to solving specific types of tasks. Generally more constrained UX — you need to know the exact inputs to the flow.	Lower successful task completion rate. More challenging to instrument, test, and evaluate.

Users don't care whether you've built a fancy agent — they want a product that works consistently. The general recommendation: **default to workflows**, and only reach for agents when they're truly required. Workflows give most production applications the reliability and predictability they need; agents trade that reliability for flexibility in scenarios where the exact requirements can't be predetermined. Reach for a workflow when the process is well-defined; reach for an agent when requests are unpredictable, varied, and need creative problem-solving.

Closing: How the Pieces Fit Together

None of the six parts above are meant to be used in isolation — a real application usually composes several of them at once:

- An **agent** (Part VI) is, mechanically, the tool-use loop from Part III — the only difference is *who decides the sequence of tool calls*: a fixed script (a workflow) or Claude itself (an agent).
- **RAG** (Part IV) is naturally exposed to an agent *as a tool* — "search the knowledge base" is just another function Claude can decide to call, the same way `get_current_datetime` is in Part III.
- **Extended thinking** (Chapter 27) is what lets an agent reason *between* tool calls in a long agentic loop, not just once at the start.
- **Prompt caching** (Chapter 31) is what keeps a long agentic loop or a RAG-heavy pipeline affordable — the system prompt and tool schemas that get

resent on every iteration of Chapter 16's `while` loop are exactly the kind of static, repeated prefix caching is built for.

- **Evaluation** (Part II) applies just as much to a workflow's individual Claude calls as to an agent's overall task success rate — you still need a way to measure "did this actually work" before you trust either one in production.
- **Vision, PDFs, and citations** (Chapters 28–30) are just richer content blocks — they slot into the exact same `messages` array from Chapter 1 without changing anything about history management, tool use, or streaming.

The throughline across every part of this book is the same: the Messages API is one small, stable primitive — a list of turns in, one turn out — and everything else (streaming, tools, RAG, vision, agents) is additional structure layered onto that same request and response shape, not a different API to learn each time.